UNIT V – Introduction to Machine Learning

AI & ML for Mechanical Engineering (23A03701)

➤ Syllabus Checklist

ML as a sub-discipline of AI · Supervised · Unsupervised · Reinforcement learning · Classification · Regression · Clustering · Neural networks & deep learning

1. What is Machine Learning?

➤ Definition (Tom Mitchell, 1997)

A program ***learns*** from experience ***E*** with respect to task ***T*** and performance measure ***P***, if its performance on T, measured by P, improves with experience E.

- ***Arthur Samuel (1959):*** ML gives computers the ability to learn without being explicitly programmed.
- Traditional programming: ***Data + Rules → Answers***
- Machine learning: ***Data + Answers → Rules (model)***

Example: Spam filter. T = classify mail as spam / not spam, E = mails labelled by users, P = % of mails classified correctly.

Mechanical example: T = predict bearing failure, E = past vibration records with failure dates, P = prediction accuracy.

➤ AI vs ML vs Deep Learning

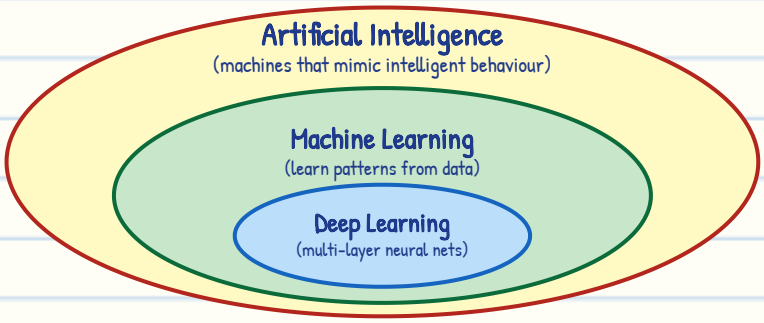
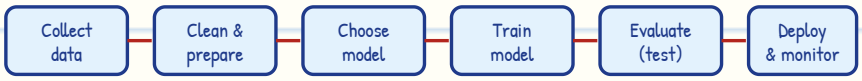


Fig 1: DL ⊂ ML ⊂ AI

➤ Why ML for Mechanical Engineers?

- Large sensor data (vibration, temperature, pressure) from machines, CNC, IoT.
- Physics-based models are slow / unknown → data-driven models are fast.
- Applications: predictive maintenance, fault diagnosis, tool-wear prediction, CFD/FEA surrogate models, material property prediction, robotics & control, additive-manufacturing quality, design optimisation.

➤ Steps in a Machine Learning Project



2. Types of Machine Learning

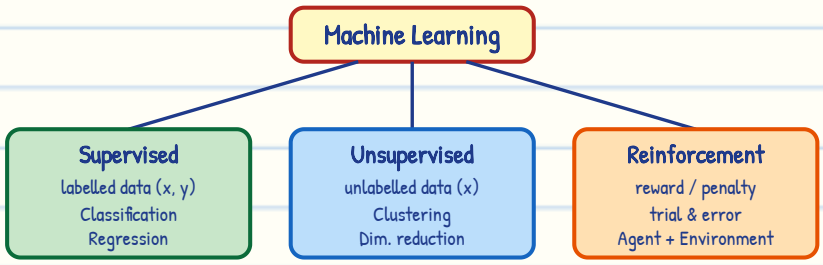


Fig 2: Classification of ML techniques

➤ 2.1 Supervised Learning

- Training data has ***inputs X and known outputs (labels) y***. A "teacher" supplies the correct answer.
- Goal: learn a function ***f: X → y*** that predicts y for new, unseen X.
- Two kinds: ***Classification*** (y = discrete class) and ***Regression*** (y = continuous value).

- Examples: fault / no-fault from vibration; predicting tensile strength from composition.
- Algorithms: Linear/Logistic regression, KNN, Decision tree, Random forest, SVM, Neural nets.

➤ 2.2 Unsupervised Learning

- Data has ***no labels***; the algorithm finds hidden structure by itself.
- Tasks: ***clustering***, association rules, dimensionality reduction (PCA), anomaly detection.
- Examples: grouping machines by operating behaviour; segmenting customers; detecting unusual sensor patterns.

➤ 2.3 Reinforcement Learning (RL)

- An ***agent*** interacts with an ***environment***, takes ***actions***, gets ***rewards***; it learns a ***policy*** that maximises total reward.
- No labelled examples – learning by trial and error. Examples: robot walking, self-driving, HVAC / engine control, game playing.

Point	Supervised	Unsupervised	Reinforcement
Data	Labelled	Unlabelled	No fixed data; feedback
Feedback	Direct (correct answer)	None	Delayed reward
Goal	Predict output	Find structure	Maximise reward
Examples	Spam, price	K-means, PCA	Robot, games

3. Classification Problems

➤ Definition

Classification predicts a ***discrete category (class label)*** for an input. Binary (2 classes: healthy/faulty) or multi-class (normal / unbalance / misalignment / bearing fault).

➤ 3.1 K-Nearest Neighbours (KNN)

- "Lazy" learner – stores all training data. For a new point: find the ***K closest*** training points (Euclidean distance) and take a ***majority vote***.
- Distance: $d = \sqrt{[(x_1-a_1)^2 + (x_2-a_2)^2]}$. Small K → noisy; large K → too smooth. Scale features first!

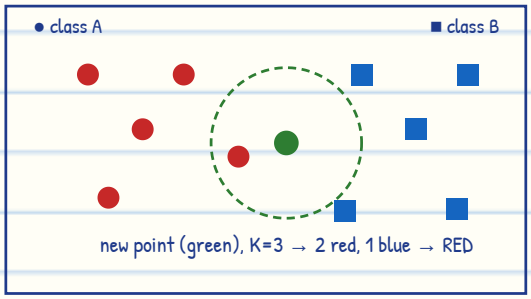


Fig 3: KNN with K = 3

➤ 3.2 Decision Tree

- Tree of if-else questions; each internal node tests a feature, leaves give the class. Easy to interpret (like a flowchart).
- Best split chosen by ***Entropy / Information Gain*** or ***Gini index***.
- Entropy $H(S) = -\Sigma\, p_i \log_2 p_i$; $Gain(S,A) = H(S) - \Sigma\, (|S_v|/|S|)\cdot H(S_v)$; $Gini = 1 - \Sigma\, p_i^2$

Example: 14 samples, 9 "Fault", 5 "OK". $H = -(9/14)\log_2(9/14) - (5/14)\log_2(5/14) = 0.410 + 0.530 = \mathbf{0.940}$. Pure node → H = 0; 50-50 node → H = 1.

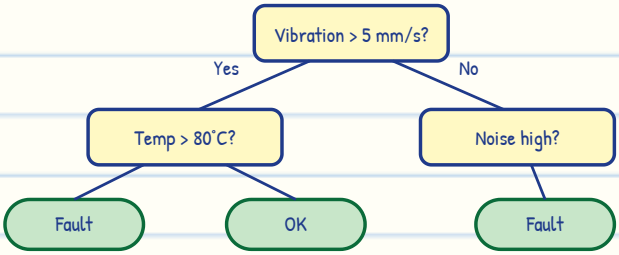


Fig 4: Decision tree for machine health

➤ 3.3 Logistic Regression

- Despite its name, used for ***classification***. Computes $z = w\cdot x + b$ and squashes it with the ***sigmoid***: $\sigma(z) = 1 / (1 + e^{-z})$ giving probability between 0 and 1.
- Rule: if $P(y=1|x) \geq 0.5$ → class 1 else class 0. Loss = binary cross-entropy: $-[y \log p + (1-y) \log(1-p)]$.
- $\sigma(0)=0.5$, $\sigma(2)=0.88$, $\sigma(-2)=0.12$. Decision boundary is a straight line ($z = 0$).

➤ 3.4 Support Vector Machine (SVM) – basics

- Finds the ***hyperplane that separates classes with maximum margin***. Points touching the margin are ***support vectors***.
- Hyperplane: $w\cdot x + b = 0$; margin width = $2/\|w\|$ → maximise margin = minimise $\|w\|$.

- **Kernel trick** (RBF, polynomial) handles non-linear data by mapping to higher dimensions. Parameter C trades margin width vs misclassification.

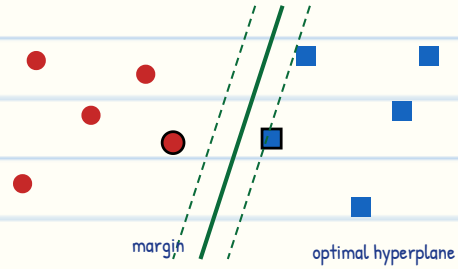


Fig 5: SVM – maximum margin (circled = support vectors)

➤ 3.5 Evaluating Classifiers – Confusion Matrix

	Predicted Positive	Predicted Negative
Actual Positive	TP (True Positive)	FN (False Negative)
Actual Negative	FP (False Positive)	TN (True Negative)

- **Accuracy** = (TP+TN)/(TP+TN+FP+FN) · **Precision** = TP/(TP+FP) · **Recall** = TP/(TP+FN)
- **F1** = 2·P·R/(P+R) (harmonic mean). Recall is vital for fault detection – a missed fault (FN) is costly.

Numerical: A fault detector tested on 100 gearboxes: TP = 40, FP = 10, FN = 5, TN = 45. Accuracy = (40+45)/100 = **85%**; Precision = 40/50 = **0.80**; Recall = 40/45 = **0.889**; F1 = 2(0.8)(0.889)/(0.8+0.889) = **0.842**.

4. Regression Problems

➤ Definition

Regression predicts a ***continuous numerical value*** (e.g., tool life in minutes, yield strength in MPa, surface roughness Ra) from input features.

➤ 4.1 Simple Linear Regression

- Model: $\hat{y} = mx + c$ (or $h\theta(x) = \theta_0 + \theta_1 x$). Multiple regression: $\hat{y} = b_0 + b_1x_1 + b_2x_2 + \dots$
- **Least squares:** choose m, c to minimise the sum of squared errors SSE = $\Sigma(y_i - \hat{y}_i)^2$.

- $m = \Sigma(x-\bar{x})(y-\bar{y}) / \Sigma(x-\bar{x})^2$ and $c = \bar{y} - m\cdot\bar{x}$

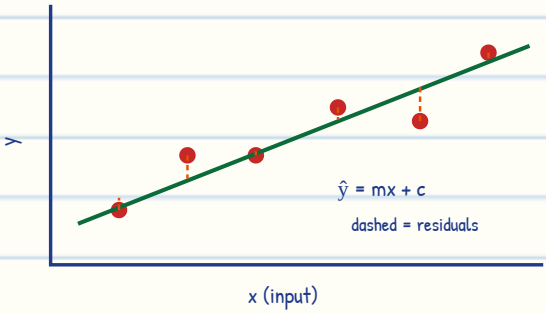


Fig 6: Best-fit line minimises squared residuals

➤ Worked Example

Data: x = 1,2,3,4,5 and y = 2,4,5,4,5. Find the line and predict y at x = 6.

$\bar{x} = 3$, $\bar{y} = 4$. $\Sigma(x-\bar{x})(y-\bar{y}) = (-2)(-2)+(-1)(0)+(0)(1)+(1)(0)+(2)(1) = \mathbf{6}$. $\Sigma(x-\bar{x})^2 = 4+1+0+1+4 = \mathbf{10}$.

$m = 6/10 = \mathbf{0.6}$, $c = 4 - 0.6\times3 = \mathbf{2.2} \rightarrow \hat{y} = \mathbf{0.6x + 2.2}$. At x = 6: $\hat{y} = \mathbf{5.8}$.

Predictions: 2.8, 3.4, 4.0, 4.6, 5.2 → residuals −0.8, 0.6, 1.0, −0.6, −0.2; SSE = 2.4, MSE = 2.4/5 = **0.48**, SST = $\Sigma(y-\bar{y})^2 = 6$, $\mathbf{R^2 = 1 - 2.4/6 = 0.6}$.

➤ 4.2 Cost Function & Gradient Descent

- Cost (MSE): $\mathbf{J(\theta) = (1/2n)\ \Sigma\ (h\theta(x_i) - y_i)^2}$ – measures how wrong the model is.
- ***Gradient descent*** – iterative minimisation: $\theta := \theta - \alpha \cdot \partial J/\partial\theta$ where α = learning rate.
- For linear regression: $\theta_0 := \theta_0 - \alpha(1/n)\Sigma(h-y)$; $\theta_1 := \theta_1 - \alpha(1/n)\Sigma(h-y)x$.
- α too small → slow; α too large → overshoots / diverges. Types: Batch, Stochastic (SGD), Mini-batch.

Mini example: $J(\theta) = \theta^2$, gradient = 2θ, $\alpha = 0.1$, start $\theta = 4$. Step 1: $\theta = 4 - 0.1(8) = \mathbf{3.2}$; step 2: $3.2 - 0.1(6.4) = \mathbf{2.56}$; → heads to 0 (minimum).

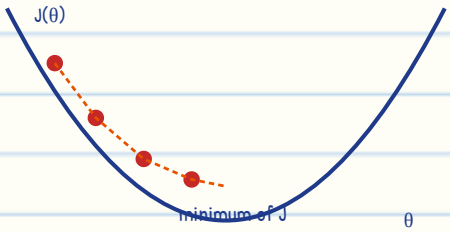


Fig 7: Gradient descent walks downhill

➤ 4.3 Other Regression Metrics

Metric	Formula	Meaning
MAE	$(1/n)\Sigma\lvert y-\hat{y}\rvert$	average absolute error
MSE	$(1/n)\Sigma(y-\hat{y})^2$	penalises big errors
RMSE	$\sqrt{MSE}$	same unit as y
R^2	1 – SSE/SST	fraction of variance explained (1 = perfect)

Other models: polynomial regression (curved data), Ridge/Lasso (regularised), regression trees, SVR, neural nets.

5. Clustering Problems

➤ Definition

Clustering groups ***unlabelled*** data so that points in the same cluster are ***similar*** and those in different clusters are ***dissimilar***. Unsupervised.

- Similarity measured by distance: Euclidean, Manhattan; or cosine similarity.
- Uses: grouping machine-operating regimes, part-family formation in group technology, image segmentation, market segmentation, anomaly detection.

➤ 5.1 K-Means Algorithm

1. Choose K (number of clusters) and pick K initial centroids.
2. ***Assign*** every point to the nearest centroid.
3. ***Update*** each centroid = mean of its points.
4. Repeat 2–3 until centroids stop changing (convergence). Objective: minimise within-cluster sum of squares (WCSS).

Worked example (1-D): points 1, 2, 4, 10, 11, 12; K = 2; initial centroids c_1 = 1, c_2 = 10.

Iteration 1: Cluster 1 = {1,2,4} (4 is 3 away from 1, 6 from 10), Cluster 2 = {10,11,12}. New c_1 = 7/3 = ***2.33***, c_2 = 33/3 = ***11***.

Iteration 2: 4 is 1.67 from 2.33 and 7 from 11 → same clusters → ***converged***. Final: {1,2,4} and {10,11,12}.

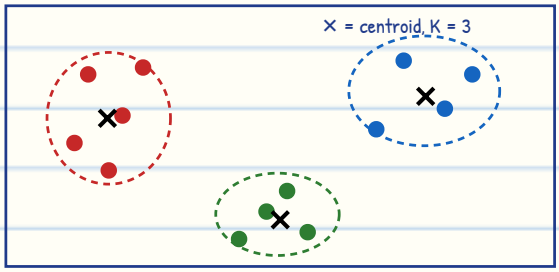


Fig 8: K-means clusters (2-D, K = 3)

- **Choosing K:** Elbow method (plot WCSS vs K; pick the bend), silhouette score.
- **Limitations:** must fix K; sensitive to initial centroids & outliers; finds only round clusters. Fix: K-means++ initialisation.

➤ 5.2 Hierarchical Clustering (basics)

- **Agglomerative (bottom-up):** start with each point as its own cluster, repeatedly merge the two closest clusters until one remains.
- **Divisive (top-down):** start with one big cluster and split.
- Linkage: single (min distance), complete (max), average. Result drawn as a **dendrogram**; cut at a height to get clusters. No need to fix K beforehand.
- Other: DBSCAN (density-based, finds arbitrary shapes, detects noise), Gaussian mixtures. Also PCA for dimensionality reduction.

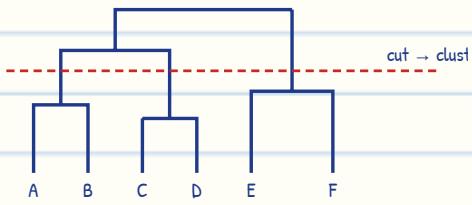


Fig 9: Dendrogram (schematic)

6. Reinforcement Learning

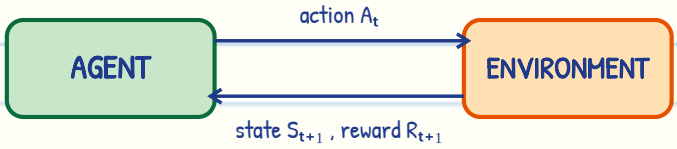


Fig 10: Agent–environment loop

Term	Meaning
Agent	The learner / decision maker (robot, controller)
Environment	Everything the agent interacts with
State (s) / Action (a)	Current situation / choice the agent makes
Reward (r)	Scalar feedback: +ve good, −ve bad
Policy (π)	Mapping state → action (the strategy)
Value / Q-value	Expected total future reward
Discount γ	0–1, importance of future rewards

- ***Exploration vs exploitation:*** try new actions (explore) or use the best known (exploit). ε-greedy: random action with probability ε.
- Modelled as a Markov Decision Process (MDP). Return $G = r_1 + \gamma r_2 + \gamma^2 r_3 + \ldots$

➤ Q-Learning (model-free)

$$Q(s,a) \leftarrow Q(s,a) + \alpha\,[\,r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)\,]$$

Example: $\alpha = 0.5$, $\gamma = 0.9$, all $Q = 0$ initially. Robot moves from s to goal s′, reward $r = 10$, $\max Q(s') = 0 \rightarrow Q(s,a) = 0 + 0.5(10 + 0 - 0) =$ ***5***.

Earlier state s_0 moves into s with $r = 0$: $Q(s_0,a) = 0 + 0.5(0 + 0.9\times5 - 0) =$ ***2.25***. Reward value "propagates backwards" along the path.

- Deep RL (DQN, PPO) uses neural networks for large state spaces. Mechanical uses: robot arm control, gait, drone control, energy management, process parameter tuning.

7. Overfitting, Underfitting & Model Evaluation

- ***Underfitting:*** model too simple – high error on both training and test data (high ***bias***).
- ***Overfitting:*** model memorises training data incl. noise – low training error, high test error (high ***variance***).
- ***Good fit:*** low error on both – generalises well.

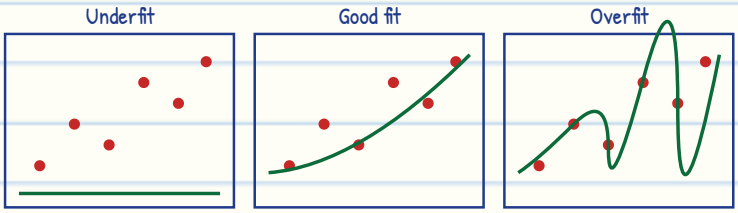


Fig 11: Three fits of the same data

➤ Train / Validation / Test Split

- Split data: ***Train (~70%)*** learns parameters; ***Validation (~15%)*** tunes hyper-parameters; ***Test (~15%)*** final unbiased check – never used in training.
- ***k-fold cross-validation:*** divide into k parts, train on k–1 and test on 1, rotate, average the scores.
- ***Prevent overfitting:*** more data, simpler model, ***regularisation*** (L1/L2), early stopping, dropout, pruning, cross-validation.

➤ Bias–Variance Trade-off

- Total error = ***Bias² + Variance + Irreducible noise***. Increasing complexity lowers bias but raises variance; aim for the sweet spot.

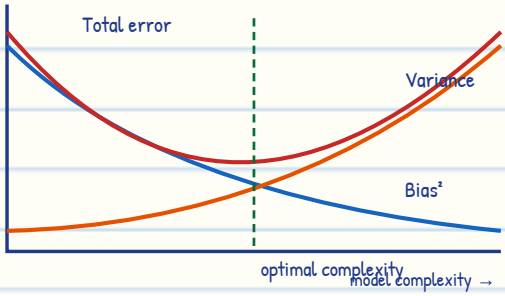


Fig 12: Bias–variance curve

8. Introduction to Neural Networks

- An ***Artificial Neural Network (ANN)*** is inspired by the brain: many simple processing units (neurons) connected by weighted links; learns by adjusting weights.
- Biological: dendrites (inputs) → soma (sum) → axon (output) ↔ Artificial: inputs x_i → weighted sum → activation → output.

➤ 8.1 The Perceptron / Artificial Neuron

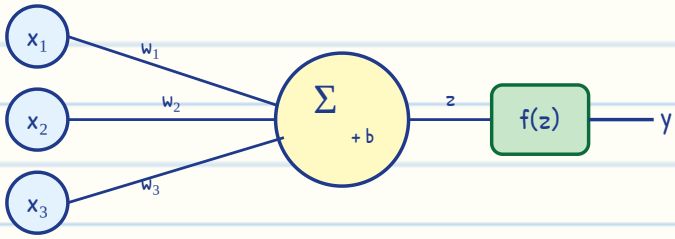


Fig 13: Artificial neuron: $y = f(\sum w_i x_i + b)$

- Weighted sum $z = \Sigma\, w_i x_i + b$; output $y = f(z)$. Weights = importance of inputs; bias b shifts the threshold.
- Perceptron (Rosenblatt 1958): step activation. Update rule: $w_i \leftarrow w_i + \eta\,(target - output)\,x_i$. Solves only ***linearly separable*** problems (AND, OR) – not XOR → need hidden layers.

> ***AND gate:*** $w_1 = w_2 = 1$, $b = -1.5$, step($z \geq 0$). Input (1,1): $z = 0.5 \rightarrow 1$. Input (1,0): $z = -0.5 \rightarrow 0$. ✔

➤ 8.2 Activation Functions

Name	Formula	Range	Use
Step	1 if $z \geq 0$ else 0	{0,1}	perceptron
Sigmoid	$1/(1+e^{-z})$	(0,1)	binary output; vanishing gradient
Tanh	$(e^{z}-e^{-z})/(e^{z}+e^{-z})$	(−1,1)	hidden layers, zero-centred
ReLU	$\max(0, z)$	$[0,\infty)$	default in deep nets, fast
Softmax	$e^{z_i}/\Sigma e^{z_j}$	probabilities sum to 1	multi-class output

➤ 8.3 Multi-Layer Network & Forward Pass

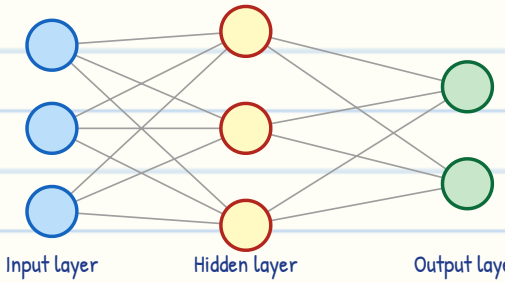


Fig 14: 3–3–2 feed-forward network

Forward pass (one neuron): $x_1 = 1$, $x_2 = 0.5$; $w_1 = 0.4$, $w_2 = 0.6$; $b = -0.1$; sigmoid. $z = 0.4(1) + 0.6(0.5) - 0.1 = \mathbf{0.6} \rightarrow y = 1/(1+e^{-0.6}) = 1/(1+0.549) = \mathbf{0.646}$. This output feeds the next layer.

➤ 8.4 Training – Backpropagation

1. **Forward pass:** compute output $\hat{y}$ from inputs; **loss** $L = \frac{1}{2}(y - \hat{y})^2$ (or cross-entropy).
2. **Backward pass:** using the **chain rule**, compute $\partial L/\partial w$ for every weight, from output layer back to input.
3. **Update:** $w \leftarrow w - \eta\, \partial L/\partial w$ (gradient descent / Adam).
4. Repeat over many **epochs** (one epoch = one pass through all data; batch = samples per update).

Chain rule idea: $\partial L/\partial w = (\partial L/\partial \hat{y})\cdot(\partial \hat{y}/\partial z)\cdot(\partial z/\partial w)$. Error signal flows **backwards** layer by layer.

- **Universal approximation:** a network with one hidden layer and enough neurons can approximate any continuous function.

9. Deep Learning

➤ Definition

Deep Learning = neural networks with **many hidden layers** that automatically learn hierarchical features from raw data (no manual feature engineering). Needs large data and GPUs.

➤ 9.1 Convolutional Neural Network (CNN)

- For **grid-like data – images** (also 2-D spectrograms of vibration signals).
- **Convolution layer:** small filters (kernels, e.g., 3×3) slide over the image to detect edges, textures. Shared weights → fewer parameters.
- **Pooling:** down-samples (max pooling) → reduces size, adds invariance. **ReLU** after convolution. **Fully connected** + softmax at the end.

- Uses: surface-defect detection, weld inspection, crack detection, thermal-image analysis, medical imaging.

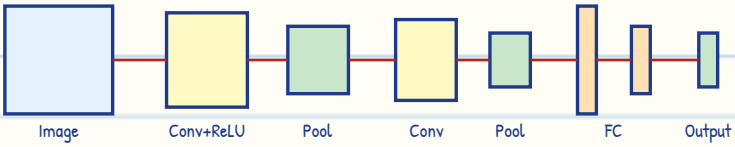


Fig 15: CNN pipeline

➤ 9.2 Recurrent Neural Network (RNN)

- For ***sequential / time-series data*** (speech, text, sensor streams). Has a ***loop***: hidden state $h_t = f(W \cdot x_t + U \cdot h_{t-1} + b)$ carries memory of previous inputs.
- Problem: ***vanishing gradient*** for long sequences → solved by ***LSTM*** and ***GRU*** (gates decide what to remember/forget).
- Uses: remaining-useful-life (RUL) prediction, vibration/time-series forecasting, speech recognition, machine translation.
- Others: Autoencoders (anomaly detection), GANs (synthetic data), ***Transformers*** (attention; LLMs), Physics-Informed Neural Networks (PINNs).

Feature	Classical ML	Deep Learning
Data needed	Small–medium	Very large
Features	Hand-crafted	Learned automatically
Hardware	CPU enough	GPU / TPU
Interpretability	Higher	"Black box"

10. Applications in Mechanical Engineering

Application	ML approach	Input → Output
Predictive maintenance	Regression, LSTM, RUL models	vibration, temperature, current → remaining life
Fault diagnosis (bearings, gears, engines)	SVM, KNN, CNN, decision tree	FFT/spectrogram features → fault class
CFD / FEA surrogate	ANN, Gaussian process, PINN	geometry & boundary conditions → flow/stress field in milliseconds

Material property prediction	Regression, random forest, ANN	composition, heat treatment → strength, hardness
Tool wear / surface finish	ANN, SVR	speed, feed, depth of cut → Ra, wear
Robotics & control	Reinforcement learning	state sensors → motor commands
Defect inspection	CNN	images → defect / no defect
Design optimisation	Surrogate + genetic algorithm	design variables → performance

Case – Predictive maintenance of a pump: sensors give vibration RMS, temperature, pressure. Steps: collect → remove noise/scale → extract features (RMS, kurtosis, FFT peaks) → train classifier (healthy/faulty) or regressor (RUL) → deploy on edge device → alert before failure. Benefit: less downtime, lower cost.

➤ Common Python Tools

- NumPy, Pandas (data) · Matplotlib (plots) · Scikit-learn (classical ML) · TensorFlow / Keras, PyTorch (deep learning).

```
from sklearn.linear_model import LinearRegression
model = LinearRegression().fit(X_train, y_train)
print(model.score(X_test, y_test))  # R² on test data
```

11. Important Exam Questions (Short Q & A)

Q1. Define Machine Learning.

Ans: Learning from experience E for task T with performance P that improves with E (Mitchell). Computers learn from data without explicit programming.

Q2. Differentiate supervised and unsupervised learning.

Ans: Supervised uses labelled data to predict outputs (classification, regression); unsupervised uses unlabelled data to find structure (clustering).

Q3. Classification vs regression?

Ans: Classification output is discrete class; regression output is continuous value.

Q4. What is overfitting? How to avoid?

Ans: Model fits noise, good on train but poor on test. Avoid by more data, regularisation, cross-validation, early stopping, simpler model.

Q5. Write the K-means steps.

Ans: Choose K → initial centroids → assign nearest → recompute means → repeat until no change.

Q6. Define precision, recall, F1.

Ans: Precision = TP/(TP+FP); Recall = TP/(TP+FN); F1 = 2PR/(P+R).

Q7. What is an activation function? Name four.

Ans: Non-linear function applied to weighted sum giving neuron output: sigmoid, tanh, ReLU, softmax (step).

Q8. Explain backpropagation briefly.

Ans: Error between output and target is propagated backwards using the chain rule to get gradients of the loss w.r.t. weights, which are updated by gradient descent.

Q9. Reinforcement learning – define agent, reward, policy.

Ans: Agent learns by interacting with environment; reward = feedback signal; policy = state-to-action strategy that maximises cumulative reward.

Q10. CNN vs RNN?

Ans: CNN – images/spatial data with convolution & pooling; RNN – sequential data with memory through recurrent hidden state (LSTM/GRU).

➤ Quick Revision

ML = data → model. ***Supervised***: classification + regression. ***Unsupervised***: clustering. **RL**: reward. $m = Sxy/Sxx$; $c = \bar{y} - m\,\bar{x}$. $\theta := \theta - \alpha\nabla J$. $Q \leftarrow Q + \alpha[r + \gamma \max Q' - Q]$. Neuron: $y = f(\Sigma wx + b)$. DL = deep NN (CNN images, RNN sequences).

★ ★ ★ *End of Unit V* ★ ★ ★